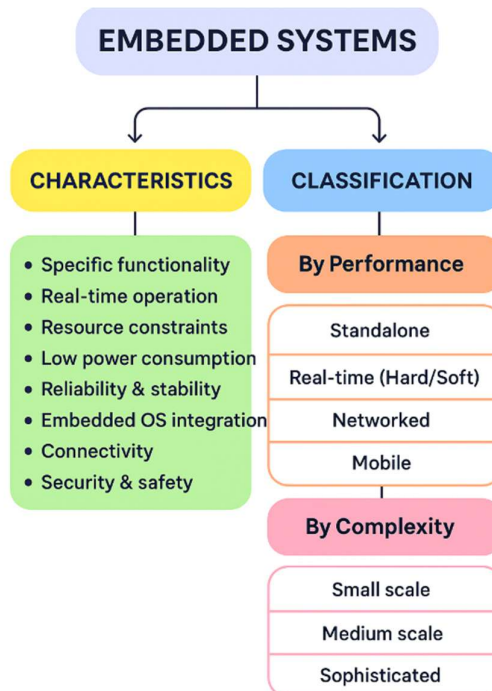


UNIT 5: EMBEDDED COMPUTING SYSTEMS: Characteristics and classification of embedded systems, Design metrics and constraints (Power, Size, Cost, Performance), Basics of Embedded Linux, Edge Computing concepts: need, architecture, examples, EdgeAI/ML: Introductory concepts, Building an IoT system with ESP32, Cloud platforms for IoT(ThingSpeak,Blynk)
Case Studies: Edge-based Predictive Maintenance, Wearable Health Monitors

Characteristics and classification of embedded systems:

An embedded system is a dedicated computer system designed for specific control functions within larger systems, often with real-time constraints. Its key characteristics include:



1. Characteristics:

1. Specific functionality: Tailored for a dedicated task, e.g., motor control, signal processing, or communication protocol handling.
2. Real-Time operation: Supports hard (strict deadline) or soft (best-effort) real-time responses, crucial for systems like pacemakers or automotive ABS.
3. Resource constraints: Limited CPU, memory, and storage to minimize cost and power usage.
4. Low Power consumption: Essential for portable and battery-operated systems like IoT devices and wearables.
5. Reliability & Stability: Must ensure fault tolerance, minimal downtime, and stable operation under varying conditions.
6. Embedded OS Integration: May run on RTOS (Real-Time Operating Systems) such as FreeRTOS, VxWorks, or embedded Linux for task scheduling and resource management.

7. Connectivity: Increasingly includes wireless interfaces (Wi-Fi, BLE, ZigBee, CAN, etc.) for IoT and automation.

8. Security & Safety: Built-in data integrity, encryption, and fail-safe mechanisms for safety-critical applications.

2. Classification of Embedded Systems

Embedded systems can be classified based on performance, function, and complexity:

A. Based on Performance and Functional Requirements

1. Standalone Systems: Operate independently, without a network connection.

Ex: Digital watches, MP3 players

2. Real-Time Systems: Deliver outputs within strict time constraints.

- Hard Real-Time – Missing deadlines leads to system failure.
- Soft Real-Time – Occasional deadline misses tolerable. Airbag controllers (Hard RT), Multimedia streaming (Soft RT)

3. Networked Systems: Connected via wired or wireless networks; support remote monitoring/control. Ex: IoT sensors, industrial PLCs

4. Mobile Embedded Systems: Designed for portable applications with mobility, often with energy constraints. Ex: Smartphones, wearables

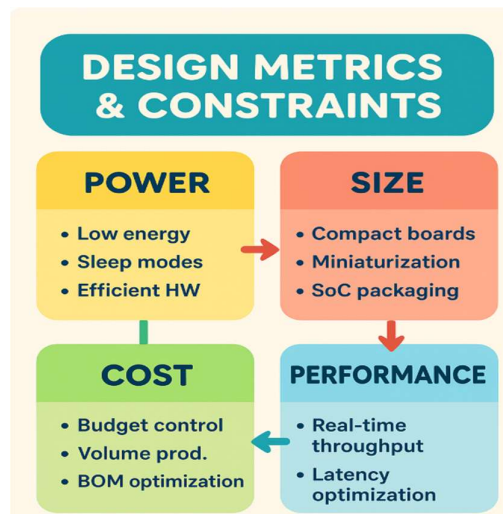
B. Based on Complexity

1. Small-Scale Embedded Systems: Use 8-bit/16-bit microcontrollers, programmed using C or assembly. simple applications with PIC, 8051 MCU-based systems.

2. Medium-Scale Embedded Systems: Use 16/32-bit microcontrollers, RTOS-based, moderate complexity. EX: ARM Cortex-M series

3. Sophisticated Embedded Systems: Multi-core processors, complex software stacks, OS-based control, often AI-enabled. Ex: Automotive ECUs, drones, network switches.

Design metrics and constraints:



Key Design Metrics:

Design metrics are the quantitative parameters that guide the design, optimization, and evaluation of embedded systems.

Metric	Technical Description	Key Considerations
1. Power	Measures energy consumption in active, idle, and sleep modes. Vital for battery-powered and IoT devices.	Power budgeting, dynamic power management (DPM), use of low-power ICs, efficient clock gating.
2. Size	Physical footprint of the hardware (PCB area, SoC package size). Smaller size improves portability and integration.	PCB layout, system-on-chip (SoC) integration, MEMS, and packaging technologies.
3. Cost	Total system cost including hardware, software, and production. Critical in consumer electronics and large-volume markets.	Trade-off between performance and affordability; economies of scale; use of off-the-shelf components.
4. Performance	Ability to meet throughput, latency, and real-time deadlines efficiently.	Choice of microcontroller architecture, optimized code, pipelining, hardware accelerators, cache tuning.

Design Constraints:

Embedded systems operate under multiple constraints that must be satisfied simultaneously:

Constraint	Explanation	Examples
Power Budget	Strict limits on energy, especially in battery-driven or wearable systems.	Smartwatches, wireless sensors
Form Factor	Restricted physical dimensions requiring compact integration.	Smartphones, IoT edge modules
Cost Sensitivity	Must be affordable for target markets; often limits high-end components.	Entry-level consumer devices
Performance Requirements	Need to deliver reliable real-time operation and desired QoS.	Automotive ECUs, network routers

Basics of Embedded Linux:

1. What is Embedded Linux?

Embedded Linux is a customized Linux operating system designed for resource-constrained embedded devices. Unlike desktop or server Linux distributions, Embedded Linux is lightweight, optimized for specific hardware, and tailored for real-time or near real-time operations.

2. Key Components of Embedded Linux

Component	Technical Description	Examples
Bootloader	Initializes hardware, loads the kernel into memory during system startup.	U-Boot, Barebox
Linux Kernel	Core of the OS, managing processes, memory, I/O, and drivers; often customized and cross-compiled for target hardware.	Linux Kernel (mainline or vendor-specific)
Root Filesystem (RootFS)	Contains user-space utilities, libraries, and applications.	BusyBox, Buildroot, Yocto-based rootfs
Device Drivers	Interfaces between hardware peripherals and the OS for communication and control.	GPIO, UART, SPI, I2C drivers
Libraries & Middleware	Provides APIs for application development.	glibc, uClibc, musl
Application Layer	User-developed applications specific to the embedded product.	Industrial control apps, IoT data processing apps

3. Development Workflow

1. Cross-Compiling – Build software for a target architecture (e.g., ARM, MIPS) on a host machine using cross-compilers (e.g., GCC-arm).
2. Kernel Configuration & Build – Configure drivers and features using tools like make menu config.
3. RootFS Creation – Assemble lightweight user space using Buildroot or Yocto Project.
4. Bootloader Setup – Flash and configure U-Boot for board initialization.
5. Deployment & Debugging – Flash image to target via TFTP/SD-card; debug using JTAG, GDB, or serial consoles.

4. Advantages of Embedded Linux

- Open-source & cost-effective
- Highly customizable kernel and user space
- Vast real time applications support and ready-to-use drivers

- Multi-tasking & multi-threading capabilities
- Suitable for real-time extensions with PREEMPT-RT patches

5. Common Embedded Linux Platforms

Hardware	Typical Use
Raspberry Pi	Prototyping, IoT gateways
BeagleBone Black	Robotics, control systems
NXP i.MX Boards	Automotive, industrial
Qualcomm Snapdragon	Consumer electronics, AR/VR

6. Popular Build Systems

Build System	Features
Buildroot	Lightweight, fast build system; ideal for small projects.
Yocto Project	Scalable and flexible; best for production-level builds.
OpenWrt	Tailored for networking devices and routers.

7. Real-Time and Embedded Linux

- PREEMPT-RT patch transforms Linux into a soft real-time OS.
- For hard real-time tasks, Linux is combined with RTOS co-kernels (Xenomai, RTLinux).

8. Applications

- IoT gateways and smart appliances
- Automotive infotainment and telematics
- Industrial automation systems
- Network devices (routers, firewalls, switches)
- Robotics and AI edge devices

Edge Computing concepts:

Edge computing is a distributed computing paradigm where data processing and storage occur closer to the source of data generation such as IoT sensors, cameras, or local gateways, instead of relying solely on centralized cloud servers.

It reduces latency, network load, and enables real-time decision-making.

Key Characteristics of Edge Computing

Characteristic	Technical Description
Proximity to Data Source	Computation happens near the data generator (e.g., IoT device, edge node).
Low Latency	Ensures ultra-fast response suitable for mission-critical applications.
Distributed Intelligence	Decentralized processing with autonomous nodes for efficiency.

Resource Optimization	Reduces cloud storage and bandwidth costs by filtering data locally.
Scalability	Supports a growing number of devices in edge networks with minimal infrastructure upgrades.
Security and Privacy	Localized processing helps secure sensitive data without sending it to public clouds.

Key Components

Component	Description
Edge Devices	IoT sensors, embedded devices that generate raw data.
Edge Nodes/Gateways	Intermediate devices with limited compute power for local analytics and filtering.
Edge AI	AI inference engines deployed locally to make predictions in real-time.
Cloud Backend	Handles deeper analytics, storage, and model training.

Benefits

- **Reduced Latency** → Immediate processing close to data source.
- **Bandwidth Optimization** → Only relevant data sent to the cloud.
- **Reliability** → Local operations remain functional even during network failures.
- **Privacy & Security** → Sensitive data remains local.
- **Real-time Decision Support** → Ideal for applications like predictive maintenance or automated control systems.

Challenges

Challenge	Explanation
Limited Compute Resources	Edge nodes have constrained CPU, memory, and power.
Complex Management	Orchestrating thousands of distributed devices is challenging.
Security Vulnerabilities	Increased attack surface due to distributed endpoints.
Interoperability	Diverse hardware and software stacks require standardized frameworks.

Applications

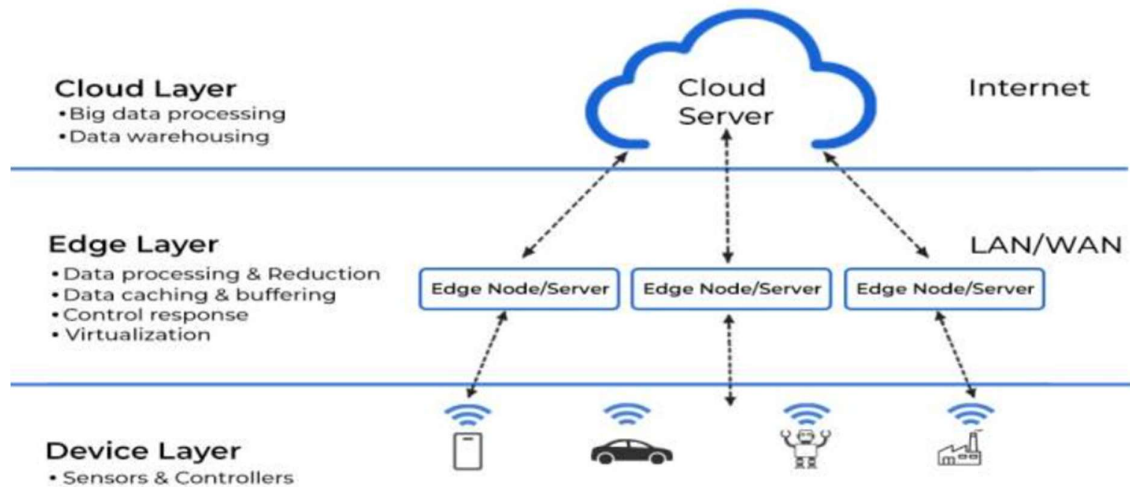
- **Industrial IoT (IIoT)** – Predictive maintenance and factory automation.
- **Autonomous Vehicles** – Local AI for navigation and safety control.
- **Smart Cities** – Real-time traffic management, surveillance, and environmental monitoring.
- **Healthcare** – Patient monitoring and diagnostics at the edge.

- **Retail** – Personalized customer experiences and inventory optimization.

Key Technologies Enabling Edge Computing

- **5G Networks** for ultra-low latency connectivity.
- **AI/ML frameworks** like TensorFlow Lite, Edge Impulse.
- **Containerization** with Docker, Kubernetes for edge deployment.
- **Hardware accelerators** (NVIDIA Jetson, Intel Movidius, Google Coral) for edge AI.

Edge computing architecture:

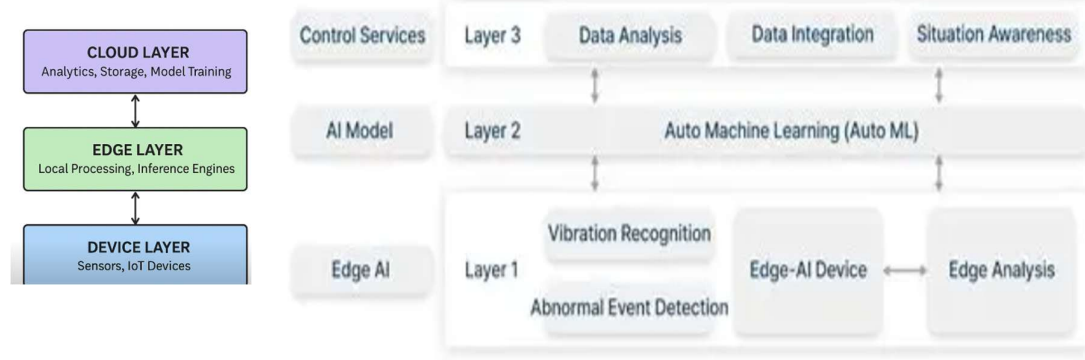


EdgeAI/ML: Introductory concepts

Edge AI/ML (Artificial Intelligence / Machine Learning at the Edge) refers to running AI models and machine learning algorithms directly on edge devices (IoT sensors, smartphones, cameras, or gateways), without heavy reliance on cloud servers. It combines edge computing with intelligent analytics to enable low-latency, secure, and efficient decision-making.

Low Latency	Eliminates delays by processing data locally instead of sending it to distant cloud servers.
Bandwidth Efficiency	Only critical data or processed summaries are sent to the cloud, saving bandwidth.
Privacy & Security	Sensitive data (healthcare or surveillance) remains local.
Offline Functionality	AI models can operate without stable internet connectivity.
Scalability	Distributed intelligence reduces the load on centralized servers.

Architecture:



Key Components of Edge AI/ML

An Edge AI system comprises several interdependent components. At the base are edge devices, such as smartphones, embedded boards like Raspberry Pi, or AI-specific hardware like the NVIDIA Jetson Nano, capable of local data processing. The AI models deployed on these devices are often optimized for efficiency, using lightweight architectures such as MobileNet, TinyYOLO, or compressed versions of larger models like BERT-Tiny. These models are executed using inference engines, frameworks specifically designed for running AI workloads efficiently on local hardware, such as TensorFlow Lite, OpenVINO, or ONNX Runtime. Supporting these layers is the communication layer, which uses protocols such as MQTT, HTTP, or WebSockets to connect edge devices to cloud servers or peer devices when data sharing or updates are necessary.

Workflow of Edge AI/ML

The typical workflow of Edge AI/ML begins with data collection, where sensors or devices capture raw data such as images, telemetry signals, or audio streams. Next, model training takes place, typically in the cloud or on high-performance servers, where datasets are large and computational resources are abundant. Once trained, the models undergo optimization processes like quantization, pruning, or conversion to edge-friendly formats to fit the memory and performance constraints of edge hardware. The optimized models are then deployed onto edge devices, where inference—the real-time execution of predictions—occurs locally. Finally, a feedback loop often sends results or data back to the cloud for further analytics, retraining, or performance tuning, enabling a continuous improvement cycle.

Edge AI offers multiple advantages. By enabling **real-time decision-making**, it is well-suited for time-sensitive applications. It significantly reduces operational costs by minimizing reliance on cloud infrastructure. Its **reliability** is especially valuable in remote environments or areas with unstable connectivity, where local computation ensures uninterrupted service. Additionally, keeping sensitive data on the device enhances **data privacy and security**.

Edge AI requires a range of hardware platforms optimized for performance and efficiency. **AI accelerators** such as Google Coral TPU, Intel Movidius, and NVIDIA Jetson series provide high-speed computation for demanding inference tasks. **Microcontrollers** with AI capabilities, like STM32 and ESP32, are suitable for ultra-low-power applications in tinyML environments. For more complex workloads, **edge gateways** powered by Qualcomm Snapdragon or NXP i.MX processors combine performance with connectivity, acting as intermediaries between devices and the cloud.

Despite its advantages, Edge AI faces several challenges. **Resource limitations** are a major factor, as edge devices often have restricted CPU performance, memory, and storage capacity. To operate effectively in such environments, AI models must undergo **optimization techniques** such as quantization and pruning or employ lightweight architectures. Additionally, the distributed nature of edge systems introduces **security vulnerabilities**, making devices more susceptible to attacks if not properly managed. **Maintenance and updates** present another challenge; managing and deploying updates across thousands of distributed devices can be complex and resource-intensive.

Edge AI differs from traditional cloud-based AI in several ways. **Latency** is significantly reduced at the edge, often in the range of milliseconds, while cloud processing introduces higher delays due to network dependencies. **Data privacy** is stronger in Edge AI, as most data is processed locally rather than stored or transmitted to external servers. However, **processing power** at the edge is limited compared to the virtually unlimited capacity of the cloud. Consequently, Edge AI is preferred for **real-time and mission-critical tasks**, while the cloud remains suitable for computationally heavy tasks like deep training or large-scale analytics.

Architecture

The architecture of Edge AI/ML typically follows a **three-layer structure**. At the bottom is the **device layer**, which includes sensors and IoT devices responsible for data generation. The middle **edge layer** performs local processing and runs inference engines, often supported by AI accelerators. At the top is the **cloud layer**, where deeper analytics, long-term storage, and model training or updates are managed. Together, these layers form a hybrid system that balances performance, latency, and scalability.

Building an IoT system with ESP32:

ESP32 is a low-cost, low-power SoC (System on Chip) with Wi-Fi and Bluetooth capabilities. It's ideal for IoT applications such as sensor networks, smart homes, wearable

devices, and automation systems. Supported by multiple environments like Arduino IDE, PlatformIO, ESP-IDF, and MicroPython.



Core Components of an IoT System

Component	Role
Microcontroller	Central control unit for sensing, processing, and communication.
Sensors & Actuators	Sensors gather data (e.g., temperature, humidity), while actuators perform actions (e.g., controlling relays, LEDs, motors).
Connectivity	Wi-Fi, BLE, or LoRa (optional modules).
Cloud/Server	For data storage, analysis, or control interfaces (Firebase, AWS IoT, ThingsBoard).
Power Supply	USB, battery, or solar-based for portability.
User Interface	Web dashboard, mobile app, or voice assistants (Alexa, Google Assistant).

Steps to Build an IoT System

Step 1: Hardware Setup

- Choose an ESP32 development board like **ESP32 DevKit v1** or **NodeMCU-ESP32**.
- Connect sensors (e.g., **DHT11** for temperature, **HC-SR04** for distance) to GPIO pins.

Step 2: Programming the ESP32

- Use **Arduino IDE** or **ESP-IDF**.
- Install ESP32 board libraries.
- Write firmware for:
 - Reading sensor data
 - Connecting to Wi-Fi
 - Sending data to the cloud via MQTT, HTTP, or WebSocket.

Step 3: Connectivity & Cloud Integration

- Set up protocols like **MQTT** for lightweight communication.
- Use platforms like:
 - **Firebase** – For real-time databases.
 - **AWS IoT Core** – For scalable solutions.
 - **ThingsBoard** – For visualization dashboards.

Step 4: Dashboard & Visualization

- Build dashboards using:
 - **Blynk** – Drag-and-drop UI for mobile apps.
 - **Node-RED** – Flow-based IoT programming.
 - **Grafana** – For analytics.

Step 5: Testing & Deployment

- Test locally with debugging tools (Serial Monitor, ESP Logs).
- Deploy to real-time scenarios ensuring **security** (SSL/TLS, OTA updates).

Design Considerations

- **Power Efficiency**
Use sleep modes to extend battery life in portable applications.
- **Data Security**
Use **encrypted communication** (e.g., HTTPS, MQTT with TLS).
- **Scalability**
Design architecture for multiple devices or future expansion.
- **Latency & Reliability**
Optimize for real-time performance with local edge processing.

Arduino Code

```
#include <WiFi.h>
#include <HTTPClient.h>
const char* ssid = "Your_SSID";
const char* password = "Your_PASSWORD";
const char* serverURL = "http://example.com/data";
void setup() {
```

```

Serial.begin(115200);
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println("WiFi Connected");
}

void loop() {
    if (WiFi.status() == WL_CONNECTED) {
        HTTPClient http;
        http.begin(serverURL);
        http.addHeader("Content-Type", "application/json");
        String jsonData = "{\"temperature\":25.3,\"humidity\":60}";
        int httpResponseCode = http.POST(jsonData);
        Serial.println(httpResponseCode);
        http.end();
    }
    delay(10000); // 10 seconds delay
}

```

Applications

- Smart home automation (lights, appliances)
- Environmental monitoring (air quality, temperature)
- Wearables and health tracking
- Industrial IoT (predictive maintenance, machine monitoring)

Sample Project Ideas

Here are some beginner-friendly project ideas for creating an IoT system using ESP32:

1. Smart Home Automation System

- Control lights and appliances from a mobile app or web dashboard. Use relay modules for powering high-voltage devices. Combine this with MQTT for effective real-time communication.
- Components Needed: ESP32, Relay Module, Blynk App (for mobile control), DHT11 sensor (for temperature/humidity).

2. Weather Station

- Build a tiny weather station to monitor environmental conditions like temperature, humidity, and light intensity, and visualize the data on a web dashboard.
- Components Needed: ESP32, BME280 sensor, LDR (light-dependent resistor).

3. Remote Plant Monitoring System

- Monitor soil moisture and control a water pump to irrigate your plants automatically.
- Components Needed: ESP32, Soil Moisture Sensor, Relay Module, and Water Pump.

4. Parking Sensor with Push Notifications

- Create a parking system that sends notifications to your phone when spaces are available using IR sensors.
- Components Needed: ESP32, IR Sensors, Blynk App for notifications.

5. Home Security System

- Implement a motion-detection system that can alert you via your mobile device when movement is detected.
- Components Needed: ESP32, PIR Sensor, Blynk App for real-time alerts.

Cloud platforms for IoT(ThingSpeak, Blynk) :

ThingSpeak

ThingSpeak is an open-source cloud platform designed for IoT data collection, analysis, and visualization. It's widely used for prototyping and research because of its simplicity and MATLAB integration.

Key Features:

- Data Collection: Easily gathers data from IoT devices via HTTP or MQTT protocols.
- Visualization: Built-in charts and graphs for real-time data visualization.
- Analytics: Integration with MATLAB for advanced analytics and custom algorithms.
- Free Tier: Allows limited channels for beginners to start learning without cost.

Common Uses:

- Weather monitoring systems
- Smart agriculture (soil moisture and temperature)
- Energy usage tracking
- Student IoT projects

Blynk

Blynk is a user-friendly IoT platform that lets you create mobile and web apps to control and monitor hardware remotely. It’s ideal for hobbyists and developers building smart home or automation projects.

Key Features:

- **Drag-and-Drop Interface:** Easy dashboard setup for beginners, no advanced coding required.
- **Multi-Protocol Support:** Works with Wi-Fi, BLE, GSM, and LoRa-enabled devices.
- **Cloud + App Integration:** Provides real-time control and notifications via smartphone apps.
- **Custom Automation:** Supports rules and triggers for advanced control logic.

Common Uses:

- Home automation (controlling lights, fans, appliances)
- Real-time sensor monitoring (temperature, humidity, motion)
- Security and surveillance systems
- IoT learning kits and STEM education

Comparison

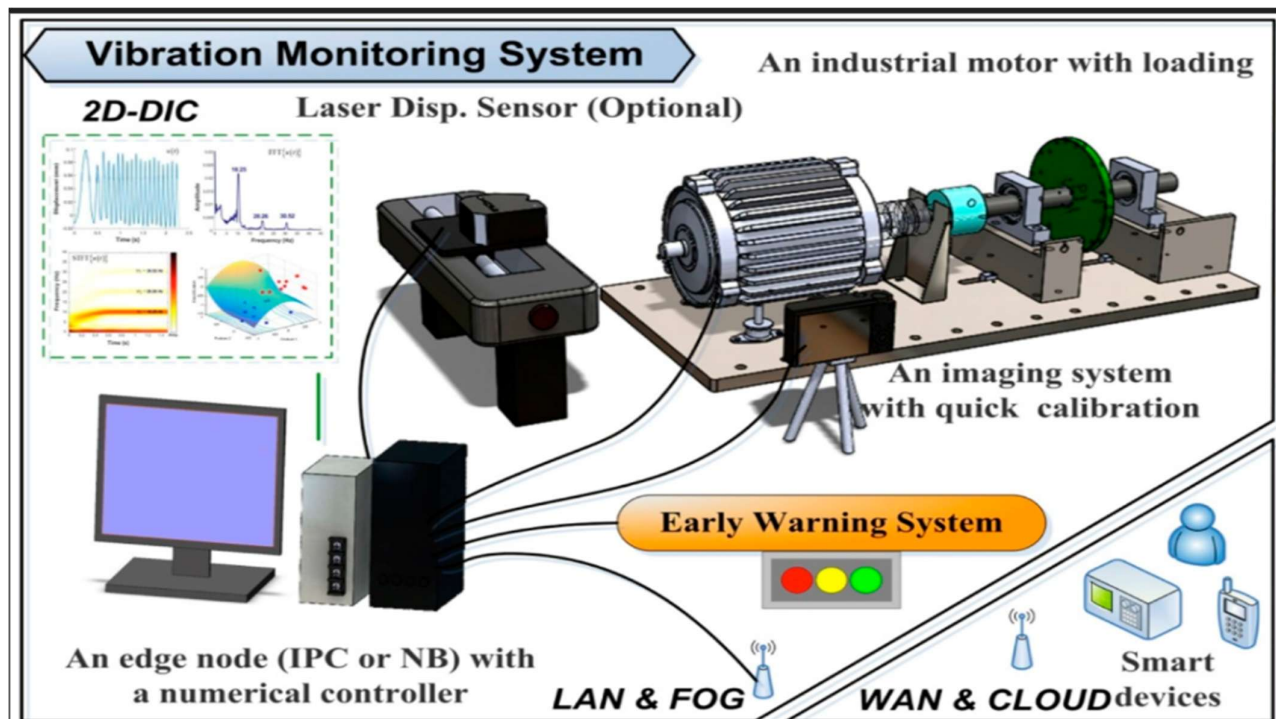
Feature	ThingSpeak	Blynk
Focus	Data logging and analytics	Remote control and dashboards
Ease of Use	Easy (requires basic coding)	Very easy (drag-and-drop UI)
Best For	Research, analytics, data visualization	Real-time control, automation
Protocols	HTTP, MQTT	HTTP, MQTT, BLE, Wi-Fi, GSM
Analytics	Strong (MATLAB support)	Limited (basic visualization)

Case Study: Edge-based Predictive Maintenance

Example Use-Case: Vibration-based Motor Health Monitoring using an Arduino + Edge AI model to detect early signs of motor failure.

Layers:

- **Device Layer:** Sensors mounted on the motor measure vibration and temperature.
- **Edge Layer:** Arduino processes signals, applies a trained AI model (via TensorFlow Lite), and detects anomalies locally.
- **Cloud Layer:** Optional — sends logs or alerts for deeper analytics and historical data storage.



Methodology Steps

Step 1: Data Collection

- Attach vibration and temperature sensors to the motor.
- Log data for normal and faulty conditions into an SD card or via serial.

Step 2: Preprocessing

- Normalize data (e.g., vibration amplitude).
- Segment readings into time windows.

Step 3: Model Training

- Use Python/TensorFlow on a PC to train a small ML model (e.g., SVM or lightweight neural network) for fault detection.
- Convert the trained model into TensorFlow Lite format for Arduino.

Step 4: Deployment

- Upload the TinyML model to Arduino using Arduino_TensorFlowLite library.

Step 5: Inference & Monitoring

- Real-time analysis at the edge.
- Trigger alerts (buzzer, LED, or cloud message) on anomaly detection.

Here's a simplified code example (no actual ML logic here, but simulating threshold-based predictive maintenance):

```
#include <Arduino.h>

// Pins

const int vibSensor = A0;
const int tempSensor = A1;
const int ledPin = 13;

// Thresholds

const float vibThreshold = 2.5;
const float tempThreshold = 60.0;

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
  Serial.println("Edge Predictive Maintenance - Motor Monitoring");
}

void loop() {
  float vibration = analogRead(vibSensor) * (5.0 / 1023.0);
  float temperature = analogRead(tempSensor) * (100.0 / 1023.0);
  Serial.print("Vibration: ");
  Serial.print(vibration);
  Serial.print(" V | Temperature: ");
  Serial.print(temperature);
  Serial.println(" C");
  if (vibration > vibThreshold || temperature > tempThreshold) {
    Serial.println("ALERT: Potential Motor Failure Detected!");
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }
  delay(1000);
}
```

Expected Results at Serial Monitor

Normal Operation:

Edge Predictive Maintenance - Motor Monitoring

Vibration: 1.8 V | Temperature: 45.0 C

System Normal

Vibration: 2.0 V | Temperature: 47.5 C

System Normal

Abnormal Condition:

Edge Predictive Maintenance - Motor Monitoring

Vibration: 3.2 V | Temperature: 62.5 C

ALERT: Potential Motor Failure Detected!

Vibration: 3.5 V | Temperature: 65.0 C

ALERT: Potential Motor Failure Detected!

Extensions Ideas:

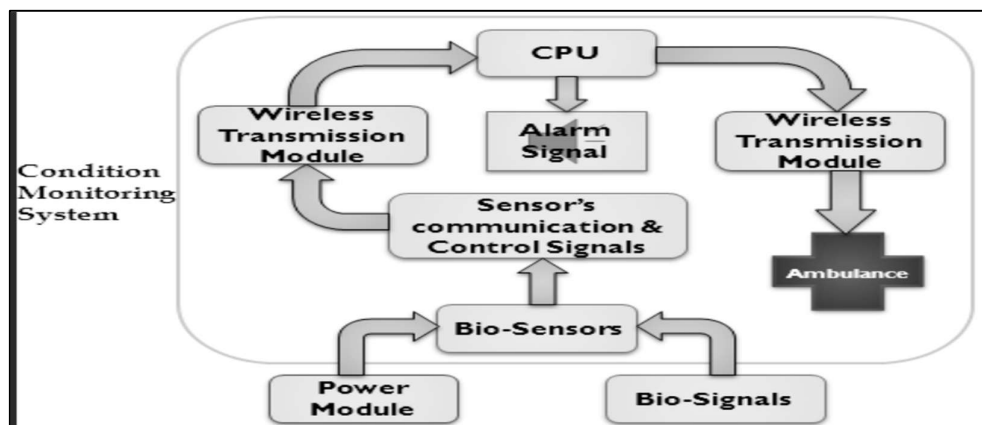
- Use TensorFlow Lite Micro for advanced ML models.
- Add wireless data transfer using ESP32 Wi-Fi or BLE to send alerts to a dashboard (like Blynk or ThingSpeak).
- Enable OTA updates to refine predictive models.

Case Study: Wearable Health Monitors

Wearable Health Monitors (Edge-AI Enabled)

- Problem: Continuous, reliable tracking of physiological and activity signals with tight power, comfort, and privacy constraints.
- Objectives:
 1. Acquire PPG (heart rate/SpO₂), ECG (optional), accelerometer/gyroscope (activity/fall), skin temperature, GSR/EDA (stress).
 2. Run on-device (edge) inference for HR, HRV, SpO₂, step count, activity class, and fall detection.
 3. Guarantee 24–72 h battery life, BLE sync, and GDPR/HIPAA-ready data handling.

System Architecture (Block Diagram)



Functional Workflow

1. **Data Acquisition:** Sensors continuously collect physiological data (heart rate, SpO₂, body temperature, ECG signals).
2. **Local Processing & AI Inference:** Raw sensor data is preprocessed and analyzed on-device by optimized deep learning models (quantized and pruned to fit microcontroller memory constraints) to detect anomalies such as arrhythmias or falls in real-time.
3. **Communication:** Processed data or alerts are transmitted wirelessly to a paired smartphone, gateway, or cloud server for further analysis, long-term storage, or remote monitoring.
4. **Visualization & Decision Support:** Users and healthcare providers access the data via mobile apps or web dashboards presenting patient status and historical trends.

- **Prototype hardware:** PPG AFE (e.g., MAX30102/04), 6-axis IMU (BMI270/ICM-42688), low-power MCU (nRF52/Cortex-M33) or ESP32 if Wi-Fi needed.
- **Data collection:** diverse cohort; label activity/falls; clinical calibration for SpO₂ (SpCO/skin tones).
- **DSP & features:** implement fixed-point filters, robust peak detection with accelerometer gating.
- **Model training:** Python → export to **TensorFlow Lite Micro/uTensor**; quantize, test on target.
- **Firmware:** RTOS tasks (sensing, inference, comms, power). Add watchdogs and brown-out detect.
- **Verification:** bench + motion profiles + treadmill; compare to FDA-cleared references.
- **Field pilot:** battery soak, BLE reliability, UX, false-alarm rate.
- **Scale:** OTA update path, fleet monitoring, privacy audits.

```
#include <Arduino.h>

// Pseudocode: replace with your sensor libs (MAX30102, ICM-42688)

const int LED_PIN = 2; // alert LED

float hr_bpm = 0, spo2 = 0;

bool fall_alert = false;

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
  // init_sensors(); init_ble(); init_tinyml_models();
  Serial.println("Wearable Monitor: PPG+IMU Edge Inference");
}

void loop() {
  // 1) Read sensors (windowed)
  // read_ppg_window(ppg_red, ppg_ir, N);
```

```

// read_imu_window(ax, ay, az, gx, gy, gz, N);
// DSP + inference (placeholders)
hr_bpm = estimate_hr_from_ppg(/*ppg_ir*/);
spo2 = estimate_spo2_ratio(/*ppg_red, ppg_ir*/);
fall_alert = detect_fall_from_imu(/*ax,ay,az,gx,gy,gz*/);
//Local decision & alert
if (fall_alert || spo2 < 92.0) {
    digitalWrite(LED_PIN, HIGH);
    Serial.println("ALERT: Fall or Hypoxemia suspected!");
} else {
    digitalWrite(LED_PIN, LOW);
}
// Summarize & emit (edge-first)
Serial.print("HR=");
Serial.print(hr_bpm, 1);
Serial.print(" bpm, SpO2=");
Serial.print(spo2, 1);
Serial.println(" %");
delay(1000); // demo; real code uses timers/RTOS
}

```

Expected Serial Output (example):

Wearable Monitor: PPG+IMU Edge Inference

HR=72.4 bpm, SpO2=98.1 %

HR=74.0 bpm, SpO2=97.6 %

ALERT: Fall or Hypoxemia suspected!

HR=110.2 bpm, SpO2=91.8 %

Validation & Pitfalls

Motion artifacts dominate errors → fuse IMU to reject corrupted PPG windows.

Skin tone & perfusion affect SpO₂ → multi-wavelength + per-user calibration.

LED power vs. battery → adaptive LED current by SNR target.

Thermal comfort → duty-cycle sensing, avoid continuous high LED currents.

Bias & generalization → stratified datasets and subgroup reporting.